

03 — Baselines, Transformers & Training

Implement the model experiments in a scientifically defensible order

Build guide for the Context-Aware Online Distress Severity Detection project.

1. Why You Need Baselines

A transformer result has little meaning without a simpler reference point.

The project should show a progression from classical NLP to increasingly specialized neural models.

Recommended progression: TF-IDF + Logistic Regression → TF-IDF + Linear SVM (optional) → generic pretrained transformer → fine-tuned RoBERTa → domain-adapted transformer.

2. Baseline 1 — TF-IDF + Logistic Regression

Convert each post into a TF-IDF vector.

Train a multiclass Logistic Regression classifier.

Evaluate it using the same held-out test protocol as the transformer.

Save the baseline metrics and confusion matrix. This becomes the reference against which all later improvements are judged.

3. Baseline 2 — Transformer

Load a pretrained RoBERTa-style sequence classification model.

Use its matching tokenizer.

Tokenize the sanitized text into input IDs and attention masks.

Fine-tune the model on the four severity classes.

4. Training Flow

```
Dataset
↓
Tokenizer
↓
Input IDs + attention masks
↓
Transformer encoder
↓
Classification head
↓
Four logits
↓
Loss function
↓
Backpropagation
↓
AdamW optimizer
↓
Updated model
```

5. Core Stack

Python, PyTorch, Hugging Face Transformers, Hugging Face Tokenizers, scikit-learn, NumPy, Pandas, Matplotlib.

Use CUDA when available, but make the code runnable on CPU for reproducibility and testing.

6. Experiment With Loss Functions

Your original description contains three ideas: weighted cross-entropy, focal loss, and a heavier penalty for underestimating severe cases.

Do not hard-code the claimed improvement. Treat loss design as an experiment.

Suggested experiment matrix:

- E1 — ordinary cross entropy.
- E2 — class-weighted cross entropy.
- E3 — focal loss.
- E4 — weighted focal loss.
- E5 — ordinal objective.

7. Severe-Class Objective

If severe examples are rare, overall accuracy can look strong while severe recall is poor.

Use class weights or another principled objective to make the severe category matter appropriately.

A '4× penalty' should be treated as a hyperparameter to test, not as an automatically correct clinical weighting.

8. Training Engineering

Use fixed random seeds where possible.

Keep training configuration outside the source code, for example in YAML files.

Log learning rate, batch size, epochs, model checkpoint, loss function, class weights, and dataset version for every experiment.

Use early stopping or checkpoint selection based on a validation metric, not the test set.

Save tokenizer and model together so inference uses exactly the same preprocessing.

9. Suggested Configuration

```
configs/  
■■■ baseline.yaml  
■■■ roberta.yaml  
■■■ domain_model.yaml
```

A configuration should control model name, max sequence length, batch size, learning rate, epochs, seed, loss type, class weights, and output directory.